
CS342- Project Description

Deadline (week 17)

Group: 4-5 students.

Description and requirement of the project:

You must develop a small application that covers multiple subjects you are studying during this course. Such as creating a graphical user interface (GUI), connecting to a database, and handling exceptions and various scenarios. The main requirements of this project are:

1. Graphical User Interface (GUI):

Your interface should contain components with actions (event handling), Example of components:

- Buttons: For navigating to different pages.
- Text Fields: To capture user input.
- Labels: To provide instructions or feedback.
- Combo Boxes and list.
- Event Handling:
 - Clicking buttons should trigger actions, such as navigating to the login or sign-up pages.
 - Text fields should validate the input (e.g., correct email format, password strength).

2. Database Integration

- Database Setup: You need to connect the application to a database where user information is stored.

3. Exception handling.

- Invalid Credentials:
 - Example: If the username or password is incorrect, display an error message.
- Input Validation:
 - Example 1: Ensure the email is valid (using a regex or built-in function).
 - Example 2: Enforce a minimum password length and other password policies.

4. Report

Finally, you must submit a report about your project that documents:

- **Application Overview:** Explain what your application does (e.g., User Authentication).
- **GUI Elements:** Describe each GUI element (button, text field, etc.) and its purpose.
- **Database Connection:** Explain how you connected the app to the database (e.g., SQLite, MySQL).
- **Exception Handling:** List the possible exceptions and scenarios you covered (e.g., invalid email format, failed login attempts, etc.).
- **Testing and Validation:** Describe how you tested the application and ensured it worked for all cases.

Proposed Projects:

You will choose one of the three proposed projects:

1. Project Title: Library Lending & Catalog System

Description: This project aims to develop a Java Swing–based Library Management System that facilitates the organization and operation of a library’s core services. The system will allow librarians to manage the library’s catalog, maintain member records, and process book lending and returns efficiently. Users will be able to search for books, check availability, request loans, and place reservation holds on borrowed items. The system will also track due dates and automatically calculate fines for overdue returns. Additionally, it will support queue-based reservation handling when multiple members request the same book. Robust input validation, database integration, and exception handling will ensure secure and reliable system functionality.

Requirements:

1. Graphical User Interface (GUI):

The application must be implemented using **Java Swing**, employing components such as `JFrame`, `JPanel`, `JTable`, `JTabbedPane`, `JTextField`, `JButton`, and `JOptionPane`. The interface should support intuitive navigation between different modules, with clear labels and user-friendly layouts.

2. Database Integration:

The system must connect to a **SQLite** or **MySQL** database using **JDBC**. The database should include at minimum the following tables:

- `books` (book metadata)
- `copies` (individual physical book units and availability status)
- `members` (member registration and credentials)
- `loans` (borrowed books and due dates)
- `fines` (overdue or penalty charges)
- `holds` (reservation queue for checked-out books)

3. User Authentication and Roles:

The system must support login access for two user roles:

- **Librarian:** full access to all system management functions
- **Member:** limited access for searching books and borrowing services

Passwords must be **securely stored** using hashing before being saved in the database.

4. Book Catalog and Search Module:

Users must be able to search and filter books by **title, author, ISBN, or category**.

Search results should display **real-time availability** based on copy status and allow viewing copy-specific details.

Pagination or scrollable tables should be used for large lists.

5. Lending and Return Management:

The system must support:

- **Borrowing** available books
- **Renewing** loans (subject to rules and hold restrictions)
- **Returning** books

Due dates, grace periods, and fines for overdue items must be automatically calculated and updated.

6. Reservation and Hold Queue System:

If all copies of a book are checked out, members should be able to **place a hold**.

When a copy becomes available, the next member in the hold queue should be notified (a **simulated in-system notification**, not email).

7. Administrative Functions (CRUD):

Librarians must be able to **add, update, and remove**:

- Book and catalog entries
- Individual book copies
- Member accounts

- Book categories / classifications

8. Reporting and Data Export:

The system must generate summary reports such as:

- List of **overdue loans**
- **Most borrowed books** within a date range
- **Active fines** owed by members

Reports should be exportable to **CSV** format.

9. Validation and Exception Handling:

The system must handle and display clear messages for cases such as:

- Duplicate ISBN or member registration
- Invalid or missing required input fields
- Maximum borrowing limit reached
- Attempting to renew overdue or held items

All database operations should include robust `try/catch` blocks with user-friendly error dialogs.

10. Testing and Verification:

The system must be tested with real-world usage scenarios, including:

- Loan renewal under overdue conditions
- Hold queue conflicts
- Returning books with fines applied

Tests should demonstrate correctness, stability, and reliability across the main workflows.

2. Project Title: Hotel Reservation & Check-In System

Description: This project aims to develop a Java Swing application that automates hotel front-desk and guest management operations. The system will allow hotel staff to manage room inventory, process reservations, handle guest check-in and check-out procedures, and generate invoices based on applied pricing rules. Seasonal pricing, promotional discounts, late checkout penalties, and refundable security deposits will be incorporated to reflect realistic hotel billing operations. The application will integrate with a relational database through JDBC and will include role-based authentication, user input validation, and robust exception handling to ensure reliable and secure operation.

Requirements:

1. Graphical User Interface (GUI):

The application must be implemented in **Java Swing**, using components such as menus, modal dialogs, date pickers, tables (`JTable`), and form panels. The system should be organized using the **MVC (Model-View-Controller)** architectural pattern to ensure clarity, maintainability, and separation of concerns.

2. Database Integration:

The system must connect to a **SQLite or MySQL** database using **JDBC**. The database shall contain, at minimum, the following tables:

- `rooms` (individual room details and attributes)
- `room_types` (category, capacity, and base pricing)
- `reservations` (guest bookings and stay durations)
- `guests` (guest personal and identification records)
- `invoices` (charges, payments, and balances)
- `payments` (payment transactions)
- `discounts` (seasonal rates, promotions, and corporate codes)

3. User Authentication and Role Management:

The system must support separate login roles:

- **Receptionist** – responsible for reservations and check-in/check-out operations
- **Manager** – full administrative privileges, including reporting and pricing control

The system should maintain **activity logs** to record which user performed each operation.

4. Reservation Management Engine:

The system must support:

- Room availability search based on **date range** and **room type**
- Automatic **conflict detection** to prevent double-booking
- A **waitlist mechanism** when no rooms are available
- Reservation modification or cancellation workflows

5. Guest Check-In and Check-Out Handling:

When checking in:

- Capture guest personal ID/passport information
- Assign room and record deposit (if applicable)

When checking out:

- Apply **late checkout fees** if needed
- Record **damages or additional service charges**
- Finalize the billing and close the reservation

6. Pricing and Discount Rules:

The system must support:

- **Seasonal pricing** adjustments
- **Promotional discount codes**
- **Corporate/organization rate agreements**
- Automatic inclusion of **taxes and service fees** in the total bill

7. Billing and Invoice Management:

The system must be able to:

- Generate itemized **invoices**
- Support **partial payments** and multi-transaction settlements
- Handle refunds when applicable
- Export or print invoices in **PDF or text** format

8. Housekeeping and Room Status Tracking:

Each room must have a status such as:

- **Available, Occupied, Cleaning Required, or Maintenance**

Conflicts must be flagged (e.g., trying to assign a dirty room).

9. Reporting and Analytics:

The system must provide summary and analytical reports such as:

- **Daily/Weekly occupancy rate**
- **Average Daily Rate (ADR)**
- **Revenue summaries**
- **Reservation cancellations and no-show statistics**

10. Validation and Exception Handling:

The system must handle and display clear error messages for:

- Invalid or overlapping date ranges
- Overbooking attempts
- Incorrect payment inputs or incomplete transaction records

All database and business-logic operations must use structured `try/catch` blocks and user-friendly message dialogs.

3. Project Title: Event Registration and Ticket Management System

Description: This project aims to develop a Java Swing application that enables users to browse upcoming events, register for events, and receive ticket confirmations. Event organizers will be able to create and manage event listings, specify seat capacities, and monitor registration statistics. Administrators will have full management control over users, events, ticket records, and reports. The system will include role-based authentication, ticket generation, event reminder notifications, and robust input validation. A relational database will be used to store event information, user accounts, seat availability, and ticket reservations, ensuring accurate and reliable data handling.

Requirements:

1. Graphical User Interface (GUI):

The application must be developed using **Java Swing**, employing components such as `JFrame`, `JPanel`, `JTabbedPane`, `JTable`, forms, menus, and dialog boxes. The interface should be intuitive, visually organized, and support smooth navigation between user roles and system modules.

2. Database Integration:

The system must use **JDBC** to connect to a **SQLite or MySQL** relational database. The database should include tables such as:

- `events` (event details: title, date, category, location, seat capacity)
- `users` (account data and role identification)
- `registrations` (user-event participation records)
- `tickets` (generated ticket IDs and status information)

3. User Authentication and Roles:

The system must support **three distinct user roles**:

- **Admin:** full system management capabilities
- **Event Organizer:** create and manage events and view attendee lists
- **Attendee:** view events and register for them

Passwords must be securely stored using hashing prior to database insertion.

4. Event Browsing and Search Filters:

Users must be able to browse and filter the event listing based on:

- Date / Time range
- Event category (e.g., workshop, seminar, concert)
- Location / Venue

Event details must display real-time seat availability.

5. Event Registration and Seat Tracking:

Attendees may register for available events.

The system must:

- Track seat availability dynamically
- Prevent overbooking and notify users when events are full
- Support cancellation of registrations where appropriate

6. Ticket Generation and Notifications:

Upon successful registration, the system must **generate a ticket** with a unique identifier.

Notification options may include:

- In-app notification dialogs
- Simulated confirmation email / message display

Ticket information must be retrievable for future reference or check-in.

7. Administrative and Organizer Dashboard:

The system must include interfaces for managing:

- Event creation, editing, and removal
- User accounts (where applicable)
- Registration statistics and attendance reports

8. Reports and Analytics:

The system must generate summary reports, such as:

- Total registrations per event
- Most attended event categories
- Capacity utilization and waitlist activity

9. Validation and Exception Handling:

The system must handle and display meaningful error messages for issues such as:

- Attempting to register for a full event
- Invalid or missing form entries
- Database connectivity or query execution errors

All operations must be enclosed in structured `try/catch` blocks with user-friendly alerts (`JOptionPane`).